

Martelli-Montanari **unifikasjonsalgoritme**

og

Hindley-Milner **typeinferens** (for basistyper)

Substitusjon og Unifikasjon

► Substitusjon

- en mengde ligninger, $\sigma = \{x_1 = t_1, \dots, x_n = t_n\}$, der variablene til venstre er forskjellige
- Anvende en substitusjon σ på en term t , $\sigma(t)$:
erstatte samtidig alle forekomster av hver x_i i t med t_i
- F.eks.: $\sigma = \{x = f(x, y)\}$, $t = g(a(x, y), x)$, $\sigma(t)$ på tavlen

► Unifikasjon ('forening') av $\{t_1 = t'_1, \dots, t_n = t'_n\}$: en substitusjon som gir syntaktisk likhet, dvs. en σ slik at $\sigma(t_1) = \sigma(t'_1), \dots, \sigma(t_n) = \sigma(t'_n)$.

- anvendelse av en substitusjon $\sigma = \{x = s\}$ på en term t , dvs., $\sigma(t)$, skrives også som $t[x/s]$.

Martelli-Montanari unifikasjonsalgoritme

- ▶ Ikke-deterministisk
 - ▶ Resultatene er ekvivalente
 - ▶ Inndata: $E = \{t_1 = t'_1, \dots, t_n = t'_n\}$
 - ▶ Utdata: en unifikasjon eller NO
 - ▶ Betingelse: utdata skal være en **mest generell unifikasjon (mgu)** av E (hver annen unifikasjon kan fås vha. substitusjon)
 - ▶ Eksempel: $\{f(X, a) = f(Y, a)\}$ unifiseres av
 - (a) $\{X = a, Y = a\} \rightsquigarrow \{f(a, a) = f(a, a)\}$
 - (b) $\{X = b, Y = b\} \rightsquigarrow \{f(b, a) = f(b, a)\}$
 - (c) $\{X = Y\} \rightsquigarrow \{f(Y, a) = f(Y, a)\}$
 - (d) $\{Y = X\} \rightsquigarrow \{f(X, a) = f(X, a)\}$
- ! Bare (c) og (d) er mgu.

Martelli-Montanari unifikasjonsalgoritme

- ▶ Inndata: $E = \{t_1 = t'_1, \dots, t_n = t'_n\}$
- ▶ Algoritme: transformer E vha. følgende regler:
 - ▶ Fjern $x = x$ og $c = c$ for variabler x og atomer c
 - ▶ Snu $f(t_1, \dots, t_k) = x$ til $x = f(t_1, \dots, t_k)$ (inkl. f en konstant)
 - ▶ Erstatt $f(t_1, \dots, t_k) = f(s_1, \dots, s_k)$ med $t_1 = s_1, \dots, t_k = s_k$
 - ▶ Hvis det finnes $x = t$ der x ikke forekommer i t ('occurs check'), erstatt alle ligninger $s = r$ (untatt $x = t$ selv) med $s[x/t] = r[x/t]$.
- ▶ Stopp med
 - NO: hvis du finner en likning der
 - occurs check feiler: $x = t$ og x forekommer i t , eller
 - $f(\dots) = g(\dots)$ og $f \neq g$ (inkl. konstanter).
 - YES: og returner transformerte E , hvis:
 1. i hver likning er venstresiden en variabel, og
 2. hver variabel som forekommer på venstresiden i en ligning ikke forekommer noe annet sted

Unifikasjon: Eksempler

1. $E, t = t \Rightarrow E$
2. $E, f(t_1 \dots t_n) = f(s_1 \dots s_n) \Rightarrow E, t_1 = s_1, \dots, t_n = s_n$
3. $E, f(t_1 \dots t_n) = g(s_1 \dots s_m) \Rightarrow NO$ $f \neq g$ or $n \neq m$
4. $E, f(t_1 \dots t_n) = x \Rightarrow E, x = f(t_1 \dots t_n)$
5. $E, x = t \Rightarrow E[x/t], x = t$ $x \notin Var(t)$
6. $E, x = t \Rightarrow NO$ $x \in Var(t)$

1. $p(X, X) = p(Y, Z)$
 $\leadsto \{X = Y, X = Z\} \leadsto \{Z = Y, X = Z\}$
 $\leadsto \{Y = Z, X = Z\} \quad p(Z, Z) = p(Z, Z)$
2. $p(X, X) = p(Y, f(Y))$ (occurs check)
 $\leadsto \{X = Y, X = f(Y)\} \leadsto \{X = Y, Y = f(Y)\}$
3. $mem(H, H:T) = mem(a, X:[b, c])$

$\leadsto \{H = a, H : T = X : [b, c]\}$	$\{H = a, H : T = d : [b, c]\}$
$\leadsto \{H = a, H = X, T = [b, c]\}$	$\{H = a, H = d, T = [b, c]\}$
$\leadsto \{X = a, H = X, T = [b, c]\}$	$\{d = a, H = d, T = [b, c]\}$
$\leadsto \{X = a, H = a, T = [b, c]\}$	
4. $mem(H, H:T) = mem(a, d:[b, c])$

Unifikasjon:

$$E, t = t \Rightarrow E$$

$$E, f(t_1 \dots t_n) = f(s_1 \dots s_n) \Rightarrow E, t_1 = s_1, \dots, t_n = s_n$$

$$E, f(t_1 \dots t_n) = g(s_1 \dots s_m) \Rightarrow \text{NO} \quad f \neq g \text{ or } n \neq m$$

$$E, f(t_1 \dots t_n) = x \Rightarrow E, x = f(t_1 \dots t_n)$$

$$E, x = t \Rightarrow E[x/t], x = t \quad x \notin \text{Var}(t)$$

$$E, x = t \Rightarrow \text{NO} \quad x \in \text{Var}(t)$$

.....

$$x = c, a = f(x, e), a = f(e, d), b = g(c, d), \quad y = f(a, b)$$

$$x = c, a = f(c, e), a = f(e, d), b = g(c, d), \quad y = f(a, b)$$

$$x = c, a = f(c, e), f(c, e) = f(e, d), b = g(c, d), \quad y = f(f(c, e), b)$$

$$x = c, a = f(c, e), c = e, e = d, b = g(c, d), \quad y = f(f(c, e), b)$$

$$x = c, a = f(c, d), c = d, e = d, b = g(c, d), \quad y = f(f(c, d), b)$$

$$x = d, a = f(d, d), c = d, e = d, b = g(d, d), \quad y = f(f(d, d), b)$$

$$x = d, a = f(d, d), c = d, e = d, b = g(d, d), \quad y = f(f(d, d), g(d, d))$$

Typeinferens (for enkle $\rightarrow$ typer)

Vi betrakter kun noen funksjonsuttrykk **og typer for disse**:

expr	::=	\exprvar $\rightarrow$ expr funapp
funapp	::=	funapp simpexpr simpexpr
simpexpr	::=	exprcon exprvar (expr) (*) ...
exprcon	::=	0 1 ... 3.14 ... True False
exprvar	::=	lower case letter

type	::=	type' $\rightarrow$ type type'
type'	::=	typecon typevar (type)
typecon	::=	Int Bool ...
typevar	::=	lower case letter

typeassignment	::=	environment exprvar :: type
environment	::=	{ typeassignment }

f.eks. $\backslash x \rightarrow (*) \times 2$:: Int $\rightarrow$ Int
 $(\backslash x \rightarrow (*) \times 2) \ 6$:: Int
 $\backslash x \rightarrow \backslash y \rightarrow x \ y$:: (a $\rightarrow$ b) $\rightarrow$ a $\rightarrow$ b

Hindley-Milners algoritme reduserer

- (1) typeinferens problemet $\Gamma \vdash \text{expr} :: t$ ($\Gamma :: \text{environment}$) til
(2) unifikasjon av et likningsystem $E(\Gamma \mid \text{expr} :: t)$ avledet fra (1).

Induksjon på formen av expr (t, a, b er typevariabler):

$$\begin{aligned} \text{(t1)} \quad E(\Gamma \mid \text{con} :: t) &= \{t = \theta(\text{con})\} \\ &\quad - \text{slå opp typen til konstanten } \text{con} \end{aligned}$$

$$\begin{aligned} \text{(t2)} \quad E(\Gamma \mid x :: t) &= \{t = \Gamma(x)\} \\ &\quad - \text{variabelen } x \text{ sjekkes i konteksten} \end{aligned}$$

$$\begin{aligned} \text{(t3)} \quad E(\Gamma \mid f \ g :: t) &= E(\Gamma \mid g : a) \cup E(\Gamma \mid f :: a \rightarrow t) \\ &\quad - a \text{ er en } \underline{\text{fersk}} \text{ typevariabel} \end{aligned}$$

$$\begin{aligned} \text{(t4)} \quad E(\Gamma \mid \lambda x \rightarrow \text{ex} :: t) &= \{t = a \rightarrow b\} \cup E(\Gamma, x :: a \mid \text{ex} :: b) \\ &\quad - a, b \text{ er } \underline{\text{ferske}} \text{ typevariabler} \end{aligned}$$

(MM) Når (t1)–(t4) terminerer,
unifiserer man den resulterende likningsmengden.

Eksempel 1

$$(t1) \quad E(\Gamma \mid con :: t) = \{t = \theta(con)\}$$

$$(t2) \quad E(\Gamma \mid x :: t) = \{t = \Gamma(x)\}$$

$$(t3) \quad E(\Gamma \mid f \ g :: t) = E(\Gamma \mid g : a) \cup E(\Gamma \mid f :: a \rightarrow t)$$

$$(t4) \quad E(\Gamma \mid \backslash x \rightarrow ex :: t) = \{t = a \rightarrow b\} \cup E(\Gamma, x :: a \mid ex :: b)$$

– a, b er ferske

For å finne typen t til $\backslash x \rightarrow x$, spør GHCi $> : t \ \backslash x \rightarrow x \dots$

$$E(\emptyset \mid \backslash x \rightarrow x :: t) =$$

$$(t4) \quad \{t = a \rightarrow b\} \cup E(x :: a \mid x :: b) =$$

$$(t2) \quad \{t = a \rightarrow b\} \cup \{b = a\} =$$

$$(MM) \quad \{t = a \rightarrow b, b = a\}$$

unifiser det resulterende ligningssystemet:

$$\{t = a \rightarrow a, b = a\}$$

dvs. svaret er $\backslash x \rightarrow x :: a \rightarrow a$

Eksempel 2: Hindley-Milners algoritme

(t1) $E(\Gamma \mid c :: \tau) = \{\tau = \theta\}$, der θ er typen til konstanten c .

(t2) $E(\Gamma \mid x :: \tau) = \{\tau = \theta\}$, der $\theta = \text{lookup}(x, \Gamma)$ for variabel x .

(t3) $E(\Gamma \mid MN :: \tau) = E(\Gamma \mid N :: a) \cup E(\Gamma \mid M :: a \rightarrow \tau)$

(t4) $E(\Gamma \mid \backslash x \rightarrow M :: \tau) = \{\tau = a \rightarrow b\} \cup E(\Gamma, x :: a \mid M :: b)$
 – a og b ferske

f.eks., $E(\emptyset \mid \backslash x \rightarrow \backslash y \rightarrow x (x y) :: \tau) =$

t4 $x :: a \mid \backslash y \rightarrow x (x y) :: b \dots\dots\dots \{\tau = a \rightarrow b\}$

t4 $x :: a, y :: c \mid x (x y) :: d \dots\dots\dots \{b = c \rightarrow d, \tau = a \rightarrow b\}$

t3 $x :: a, y :: c \mid x :: e \rightarrow d \dots\dots\dots \{b = c \rightarrow d, \tau = a \rightarrow b\}$

$\cup x :: a, y :: c \mid x y :: e \dots\dots\dots \{b = c \rightarrow d, \tau = a \rightarrow b\}$

t2 $\dots\dots\dots \{a = e \rightarrow d, b = c \rightarrow d, \tau = a \rightarrow b\}$

$\cup$ t3 $x :: a, y :: c \mid x :: f \rightarrow e \dots\dots\dots \{b = c \rightarrow d, \tau = a \rightarrow b\}$

$\cup x :: a, y :: c \mid y :: f \dots\dots\dots \{b = c \rightarrow d, \tau = a \rightarrow b\}$

$= \dots\dots\dots \{f = c, a = f \rightarrow e, a = e \rightarrow d, b = c \rightarrow d, \tau = a \rightarrow b\}$

Unifikasjon: ($\backslash x \rightarrow \backslash y \rightarrow x (x y)$)

$E, t = t$	$\Rightarrow E$	
$E, f(t_1 \dots t_n) = f(s_1 \dots s_n)$	$\Rightarrow E, t_1 = s_1, \dots, t_n = s_n$	
$E, f(t_1 \dots t_n) = g(s_1 \dots s_m)$	$\Rightarrow \perp$	$f \not\equiv g \vee n \neq m$
$E, f(t_1 \dots t_n) = x$	$\Rightarrow E, x = f(t_1 \dots t_n)$	
$E, x = t$	$\Rightarrow E[x/t], x = t$	$x \notin \text{Var}(t)$
$E, x = t$	$\Rightarrow \perp$	$x \in \text{Var}(t)$

$f = c, a = f \rightarrow e, a = e \rightarrow d, b = c \rightarrow d,$	$\tau = a \rightarrow b$
$f = c, a = c \rightarrow e, a = e \rightarrow d, b = c \rightarrow d,$	$\tau = a \rightarrow b$
$f = c, a = c \rightarrow e, c \rightarrow e = e \rightarrow d, b = c \rightarrow d,$	$\tau = (c \rightarrow e) \rightarrow b$
$f = c, a = c \rightarrow e, c = e, e = d, b = c \rightarrow d,$	$\tau = (c \rightarrow e) \rightarrow b$
$f = c, a = c \rightarrow d, c = d, e = d, b = c \rightarrow d,$	$\tau = (c \rightarrow d) \rightarrow b$
$f = d, a = d \rightarrow d, c = d, e = d, b = d \rightarrow d,$	$\tau = (d \rightarrow d) \rightarrow b$
$f = d, a = d \rightarrow d, c = d, e = d, b = d \rightarrow d,$	$\tau = (d \rightarrow d) \rightarrow d \rightarrow d$

$\backslash x \rightarrow \backslash y \rightarrow x (x y) :: (d \rightarrow d) \rightarrow d \rightarrow d$

Eksempel 3

- (t1) $E(\Gamma \mid \text{con} :: t) = \{t = \theta(\text{con})\}$
- (t2) $E(\Gamma \mid x :: t) = \{t = \Gamma(x)\}$
- (t3) $E(\Gamma \mid f\ g :: t) = E(\Gamma \mid g : a) \cup E(\Gamma \mid f :: a \rightarrow t)$
- (t4) $E(\Gamma \mid \backslash x \rightarrow ex :: t) = \{t = a \rightarrow b\} \cup E(\Gamma, x :: a \mid ex :: b)$
– a, b er ferske

For å finne typen t til $\backslash x \rightarrow x\ x$, spør GHCi $> : t\ \backslash x \rightarrow x\ x \dots$

$E(\ \emptyset \mid \backslash x \rightarrow x\ x :: \tau) =$

t4 $x :: a \mid x\ x :: b \dots \dots \dots \{ \tau = a \rightarrow b \}$
 t3 $x :: a \mid x :: c \dots \dots \dots \{ \tau = a \rightarrow b \}$
 $\cup\ x :: a \mid x :: c \rightarrow b \dots \dots \dots \{ \tau = a \rightarrow b \}$
 t2 $x :: a \mid x :: c \dots \dots \dots \{ c = a, \tau = a \rightarrow b \}$
 $\cup\ x :: a \mid x :: c \rightarrow b \dots \dots \dots \{ a = c \rightarrow b, \tau = a \rightarrow b \}$
 $= \dots \dots \dots \{ c = a, a = c \rightarrow b, \tau = a \rightarrow b \}$

Unifikasjon for $\backslash x \rightarrow x\ x$

$$E, t = t \Rightarrow E$$

$$E, f(t_1 \dots t_n) = f(s_1 \dots s_n) \Rightarrow E, t_1 = s_1, \dots, t_n = s_n$$

$$E, f(t_1 \dots t_n) = g(s_1 \dots s_m) \Rightarrow \perp \quad f \not\equiv g \vee n \neq m$$

$$E, f(t_1 \dots t_n) = x \Rightarrow E, x = f(t_1 \dots t_n)$$

$$E, x = t \Rightarrow E[x/t], x = t \quad x \notin \text{Var}(t)$$

$$E, x = t \Rightarrow \perp \quad x \in \text{Var}(t)$$

$$\text{MM}(\{c = a, a = c \rightarrow b, \tau = a \rightarrow b\}) = \\ \{c = a, a = a \rightarrow b, \tau = a \rightarrow b\}$$

og svaret er $\perp$ –

Occurs check: cannot construct the infinite type $a \sim a \rightarrow b$

$$\backslash x \rightarrow x\ x :: \perp$$

Selvapplikasjon er umulig i Haskell – den har ikke noen type.

Utvidelse til andre typer

$$(t1) \quad E(\Gamma \mid \text{con} :: t) = \{t = \theta(\text{con})\}$$

$$(t2) \quad E(\Gamma \mid x :: t) = \{t = \Gamma(x)\}$$

$$(t3) \quad E(\Gamma \mid f \ g :: t) = E(\Gamma \mid g : a) \cup E(\Gamma \mid f :: a \rightarrow t)$$

$$(t4) \quad E(\Gamma \mid \backslash x \rightarrow \text{ex} :: t) = \{t = a \rightarrow b\} \cup E(\Gamma, x :: a \mid \text{ex} :: b)$$

$$(t5) \quad E(\Gamma \mid (\text{ex1}, \text{ex2}) :: t) = \{t = (a, b)\} \\ \cup E(\Gamma \mid \text{ex1} :: a) \cup E(\Gamma \mid \text{ex2} :: b)$$

$$(t6) \quad E(\Gamma \mid x : xs :: t) = \{t = [a]\} \cup E(\Gamma \mid x :: a) \cup E(\Gamma \mid xs :: [a])$$

$$(t7) \quad E(\Gamma \mid [] :: t) = \{t = [a]\}$$

a, b er ferske overalt

$$\begin{aligned} E(\emptyset \mid [1, 2] :: t) &= E(\emptyset \mid 1 : 2 : [] :: t) = \\ &\{t = [a]\} \cup E(1 :: a) \cup E(2 : [] :: [a]) = \\ &\{t = [a]\} \cup \{a = \theta(1)\} \cup \{[a] = [b]\} \cup \{2 :: b\} \cup E([] : [b]) = \\ &\{t = [a]\} \cup \{a = \theta(1)\} \cup \{[a] = [b]\} \cup \{b = \theta(2)\} \cup \{[b] = [c]\} = \\ &\{t = [Int], \quad a = Int, \quad [Int] = [Int], \quad b = Int, \quad [Int] = [Int]\} \end{aligned}$$

... og polymorfe typer ...

► $E(\emptyset \mid \backslash x \rightarrow x :: s)$ $\{s = a \rightarrow b\}$

= $E(x :: a \mid x :: b)$ $\{b = a, s = a \rightarrow b\}$

so $\backslash x \rightarrow x :: a \rightarrow a$, siden $\{b = a, s = a \rightarrow a\}$

► let id = $\backslash x \rightarrow x$ in (id₁ 5, id₂ True)

► id₁ :: Int -> Int

id₂ :: Bool -> Bool

► Typeavledning i omgivelsen:

$E(\text{id} :: a \rightarrow a \mid (\text{id } 5, \text{id } \text{True}) :: t)$

feiler, fordi a ikke kan instansieres samtidig til Int og Bool.

► Begge id'ene kan dog betraktes som instanser av en og samme **polymorfe** typen. Avledning må skille mellom

– en type a , som kan forekomme i andre deluttrykk, og

– en typevariabel som kan instansieres vilkårlig, betegnet $\forall a$

s.a. $E(\text{id} :: \forall a . a \rightarrow a \mid (\text{id } 5, \text{id } \text{True}) :: t)$ kan lykkes

...f.eks, Read/Show :t \x -> read (show x) ?

$$\Gamma = \{\text{read} :: \text{Read } r \Rightarrow \text{String} \rightarrow r, \text{show} :: \text{Show } s \Rightarrow s \rightarrow \text{String}\}$$

$$= E(\Gamma \mid \backslash x \rightarrow \text{read} (\text{show } x) :: t)$$

$$= E(\Gamma, x :: a \mid \text{read} (\text{show } x) :: b) \quad \cup \{t = a \rightarrow b\}$$

$$= E(\Gamma, x :: a \mid (\text{show } x) :: c) \quad \cup \{t = a \rightarrow b\}$$

$$\cup E(\Gamma, x :: a \mid \text{read} :: c \rightarrow b)$$

$$= E(\Gamma, x :: a \mid x :: d) \cup E(\Gamma, x :: a \mid \text{show} :: d \rightarrow c)$$

$$\cup E(\Gamma, x :: a \mid \text{read} :: c \rightarrow b) \quad \cup \{t = a \rightarrow b\}$$

$$= (\text{Show } s, \text{Read } r) \Rightarrow$$

$$\{d = a, s \rightarrow \text{String} = d \rightarrow c, \text{String} \rightarrow r = c \rightarrow b, t = a \rightarrow b\} =$$

$$\{d = a, s = d, \text{String} = c, r = b, t = a \rightarrow b\} =$$

$$\{d = a, s = a, \text{String} = c, r = b, t = a \rightarrow b\} =$$

$$\{d = s, a = s, c = \text{String}, b = r, t = s \rightarrow r\} =$$

$$\backslash x \rightarrow \text{read} (\text{show } x) :: (\text{Show } s, \text{Read } r) \Rightarrow s \rightarrow r$$

Entydiggjøring av polymorfisme kan bli nødvendig

```
:t \x -> read (show x) :: (Show s, Read r) => s -> r
```

```
:t 2 :: Num t => t
```

og Num er en subklasse av Show

```
:t read (show 2) :: Read r => r
```

```
> read (show 2)
```

*** Exception: Prelude.read: no parse

(GHCi vet ikke til hvilken type strengen "2" skal konverteres)

```
> read (show 2) :: Int
```

2

```
> read (show 2) :: Float
```

2.0

$[\text{tail}, (+1)] :: \text{Num } [a] \Rightarrow [[a] \rightarrow [a]]$

- ▶ $\text{tail} :: [a] \rightarrow [a]$
 $(+1) :: \text{Num } b \Rightarrow b \rightarrow b$
- ▶ $E([\text{tail}, (+1)] :: t) = \{t=[c]\} \cup$
 $\{ E(\text{tail} :: c) \} \cup \{ E([(+1)] :: [c]) \}$
- ▶ $\{t=[c], c = [a] \rightarrow [a], (+1) :: c\}$
- ▶ $\{t=[c], c = [a] \rightarrow [a], \text{Num } b \Rightarrow b \rightarrow b = c\}$
- ▶ $\{t=[c], c = [a] \rightarrow [a], \text{Num } b \Rightarrow b \rightarrow b = [a] \rightarrow [a]\}$
- ▶ $\{t=[c], c = [a] \rightarrow [a], \text{Num } b \Rightarrow b \rightarrow b = [a] \rightarrow [a],$
 $\text{Num } [a], \text{Num } b \Rightarrow b = [a]\}$
- ▶ $\{\text{Num } a \Rightarrow t = [[a] \rightarrow [a]],$
 $c = [a] \rightarrow [a], \text{Num } b \Rightarrow b \rightarrow b = [a] \rightarrow [a],$
 $\text{Num } [a], \text{Num } b \Rightarrow b = [a]\}$

Kinds – er “typer” av typekonstruktorer

> :k Show

Show :: * -> Constraint

> :k Int

Int :: *

> :k Char

Char :: *

Alle typer (uten noen parametere) har kind *

> :k Maybe

Maybe :: * -> *

► data Set a = Tom | En a | Un (Set a) (Set a)

> :k Set

Set :: * -> *

> :k []

[] :: * -> *

> :t []

[] :: [a]

Oppsummering av notasjon

σ (sigma) betegner typisk en substitusjon

$\sigma(s)$ betegner anvendelse av substitusjon σ på termen s – hvis $\sigma = \{x = t\}$, betyr $\sigma(s)$ substitusjon av t for alle forekomster av x .

$s[x/t]$ betegner applikasjon av en anonym substitusjon $\{x = t\}$ på s .

$\sigma(E)$ (eller $E[x/t]$) betegner anvendelse av substitusjonen σ (eller $\{x = t\}$) på E , hva enn E er, f.eks. hvis E er et system av likninger, anvender man σ (eller $\{x = t\}$) på alle termer i alle likninger.

Γ (stor gamma) betegner omgivelse, der forskjellige variabler er gitt bestemte typer

$\Gamma \vdash \text{expr} :: t$ betyr at fra omgivelsen Γ kan vi avlede at typen til expr er t

$\vee$ - er boolsk 'eller', dvs. $A \vee B$ betyr 'A eller B'